大模型推理服务(II)
推测性解码

王宏志
哈尔滨工业大学
计算学部 海量数据计算研究中心
wangzh@hit.edu.cn
http://homepage.hit.edu.cn/wang

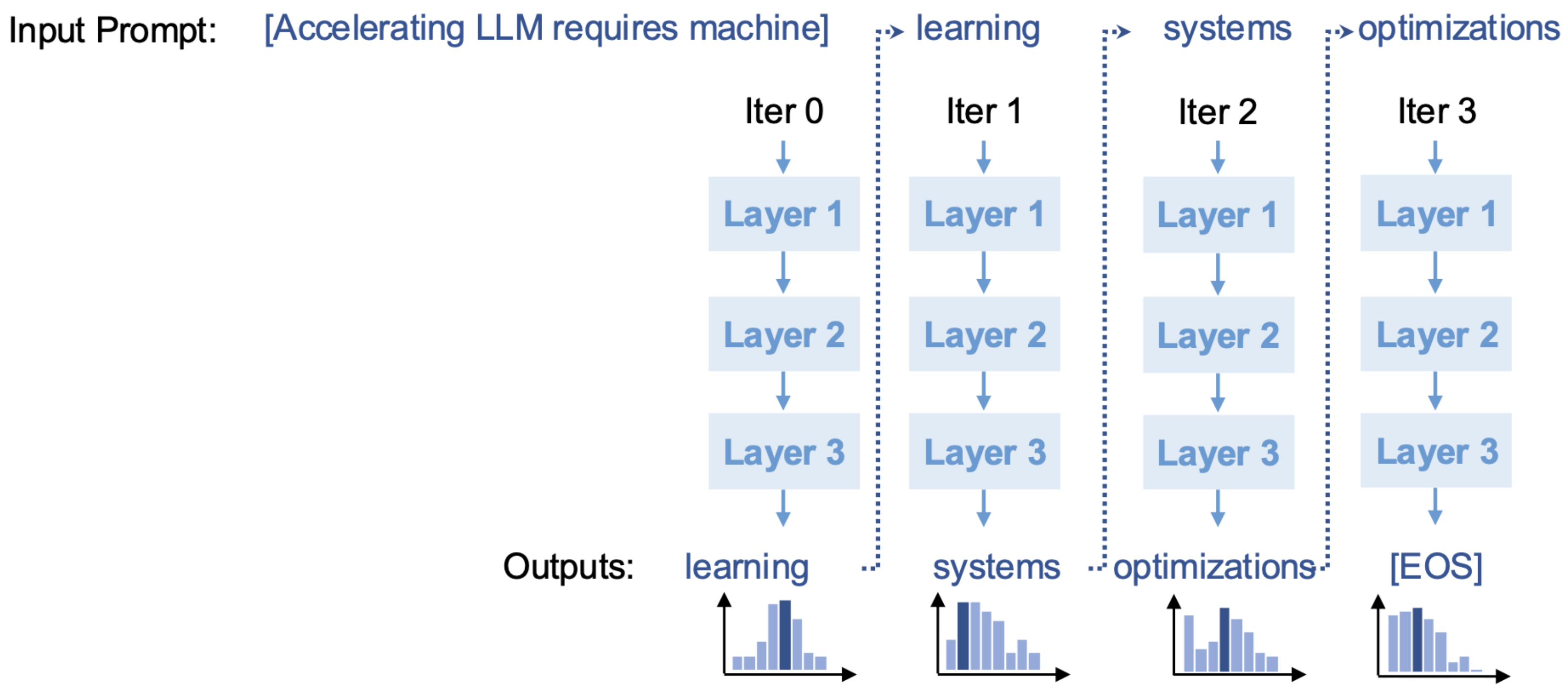
目 录

Contents

目 录

Contents

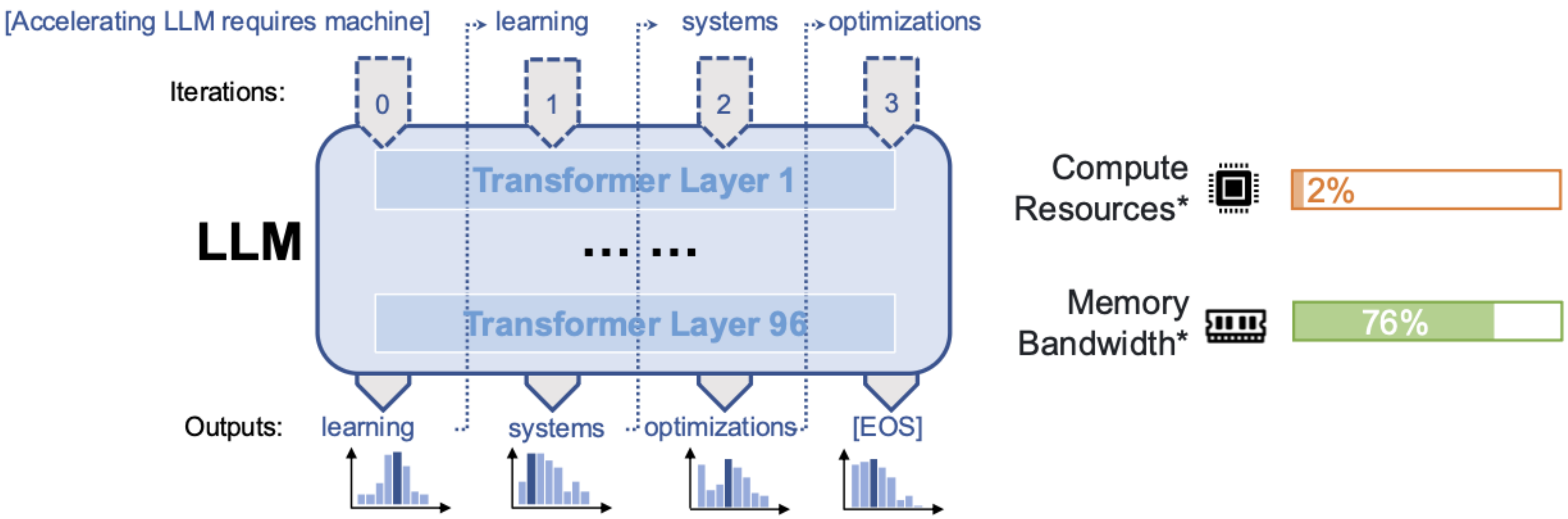
生成式 LLM 推理:自回归解码



LLM 服务缓慢且昂贵

- 至少十个 A100-40GB GPU,以半精度服务 175B GPT-3
- 生成 256 个token需要 ~20 秒
- 不能并行处理许多请求
    - 每请求键值缓存占用 3GB GPU 内存

回忆:增量解码问题



- Limited degree of parallelism → underutilized GPU resources
- Need all parameters to decode a token → bottlenecked by GPU memory access

* Measured by serving LLAMA-2-70B on 4 A100 GPUs with 4K sequence length

不同语言模型之间的权衡

# Parameters	175B	13B	2.7B	760M	125M
TriviaQA	**71.2**	57.5	42.3	26.5	6.96
PIQA	**82.3**	79.9	75.4	72.0	64.3
SQuAD	**64.9**	62.6	50.0	39.2	27.5
latency	20 s	7.6s	2.7s	1.1s	**0.3s**
# A100s	10	1	1	1	**1**

Comparing multiple GPT-3 models*

Large models

- Pro:更好的生成性能
- Con:供应缓慢且昂贵

Small models

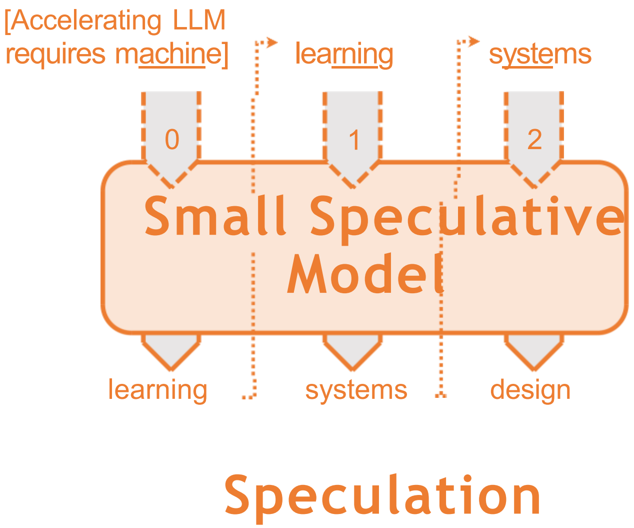
- Pro:便宜又快
- Con:准确度较低

* Language Models are Few-Shot Learners. Arxiv. 2005.14165

推测解码

1.使用小型推测模型（SSM）来预测LLM的输出

- SSM运行速度远快于LLM

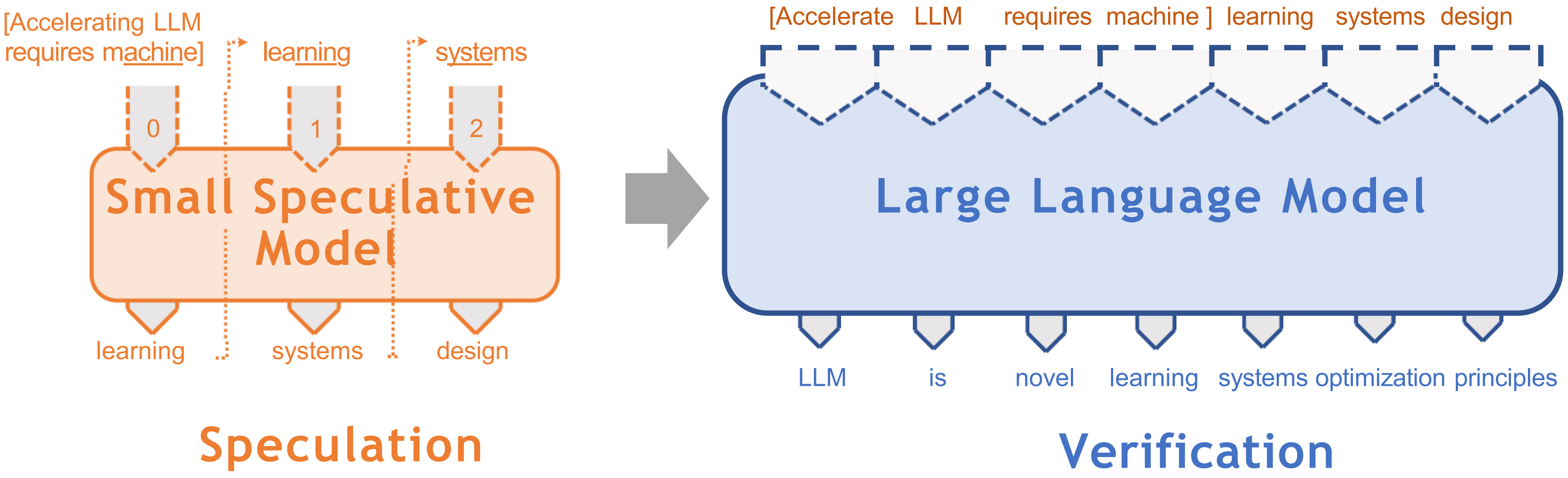


推测解码

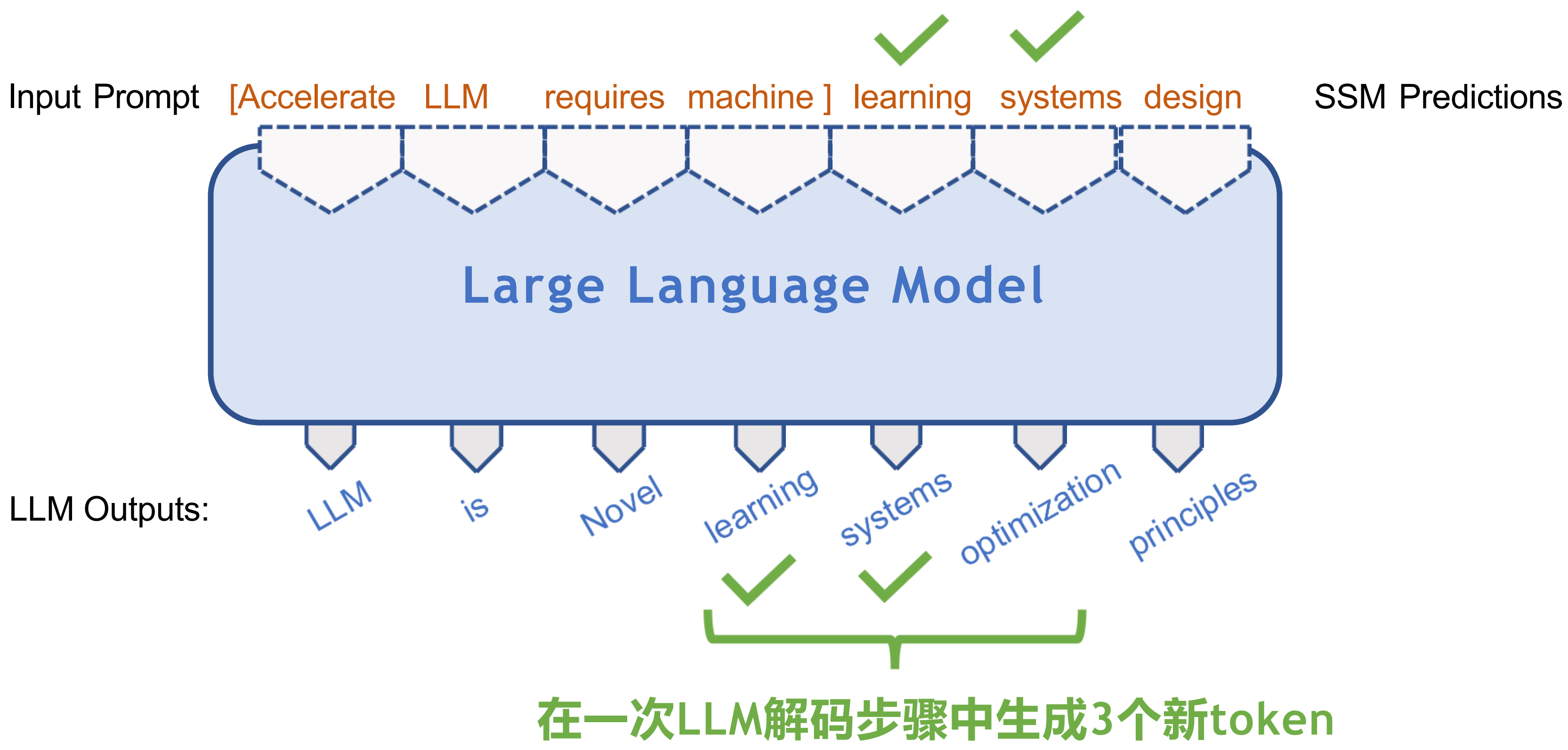
1.使用小型推测模型（SSM）来预测LLM的输出

- SSM运行速度远快于LLM

2.使用LLM验证SSM的预测

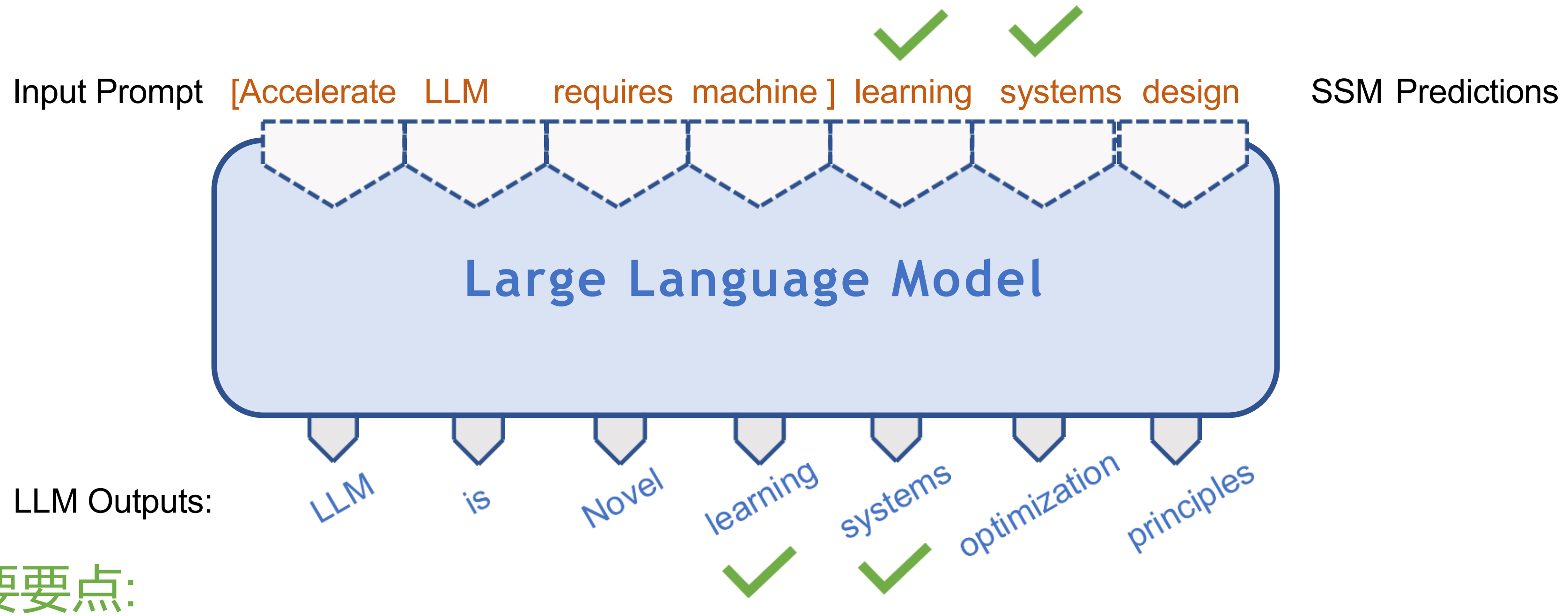


验证推测性解码结果



Input Prompt
[Accelerate LLM requires machine]
learning systems design
SSM Predictions
Large Language Model
LLM Outputs:
LLM is Novel learning systems optimization principles
在一次LLM解码步骤中生成3个新token

验证推测性解码结果



主要要点:

- LLM推断通过访问模型权重而成为瓶颈

•利用LLM解码多个token以提升GPU利用率

SpecInfer：基于树的推测推理与验证

关键思想:不是把LLM当作增量解码器，而是用它们作为**并行token树验证器**

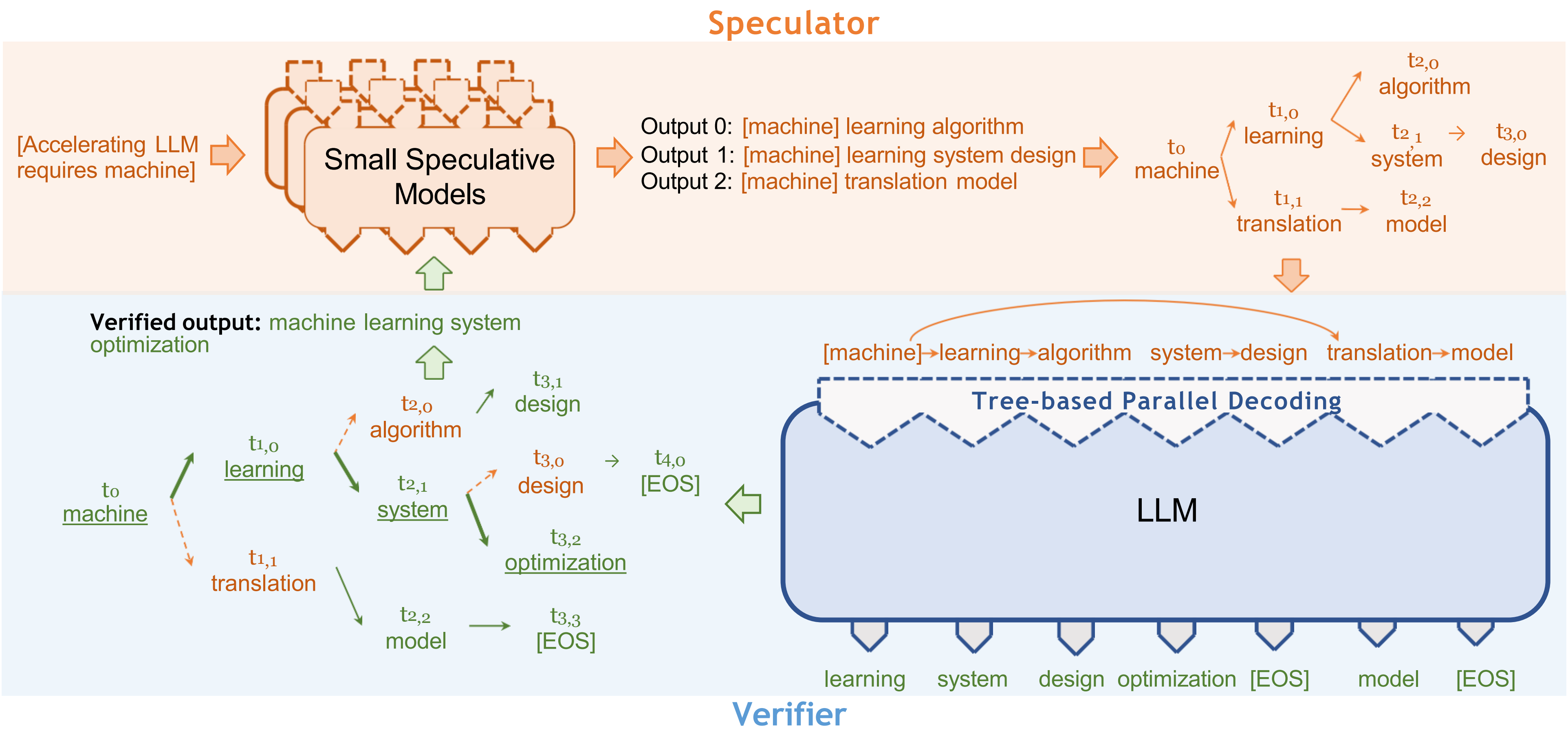
•性能提升 :优于现有的大型语言模型系统1.3-2.4x

•更高效率： 2.5-4.4x

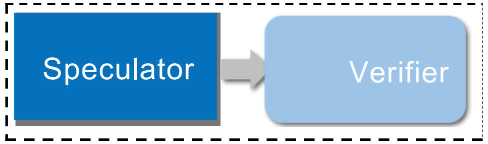
•正确性： 验证保证端到端等价

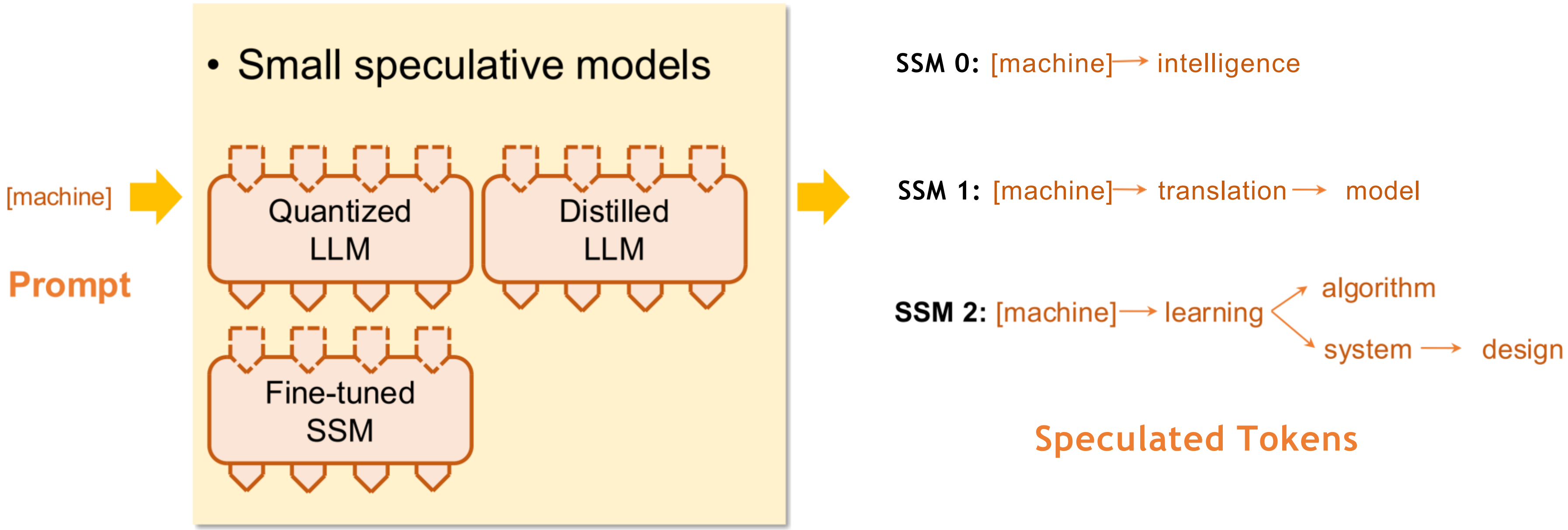
* SpecInfer: Accelerating LLM Serving with Tree-based Speculative Inference and Verification. ASPLOS’24

SpecInfer 工作流程

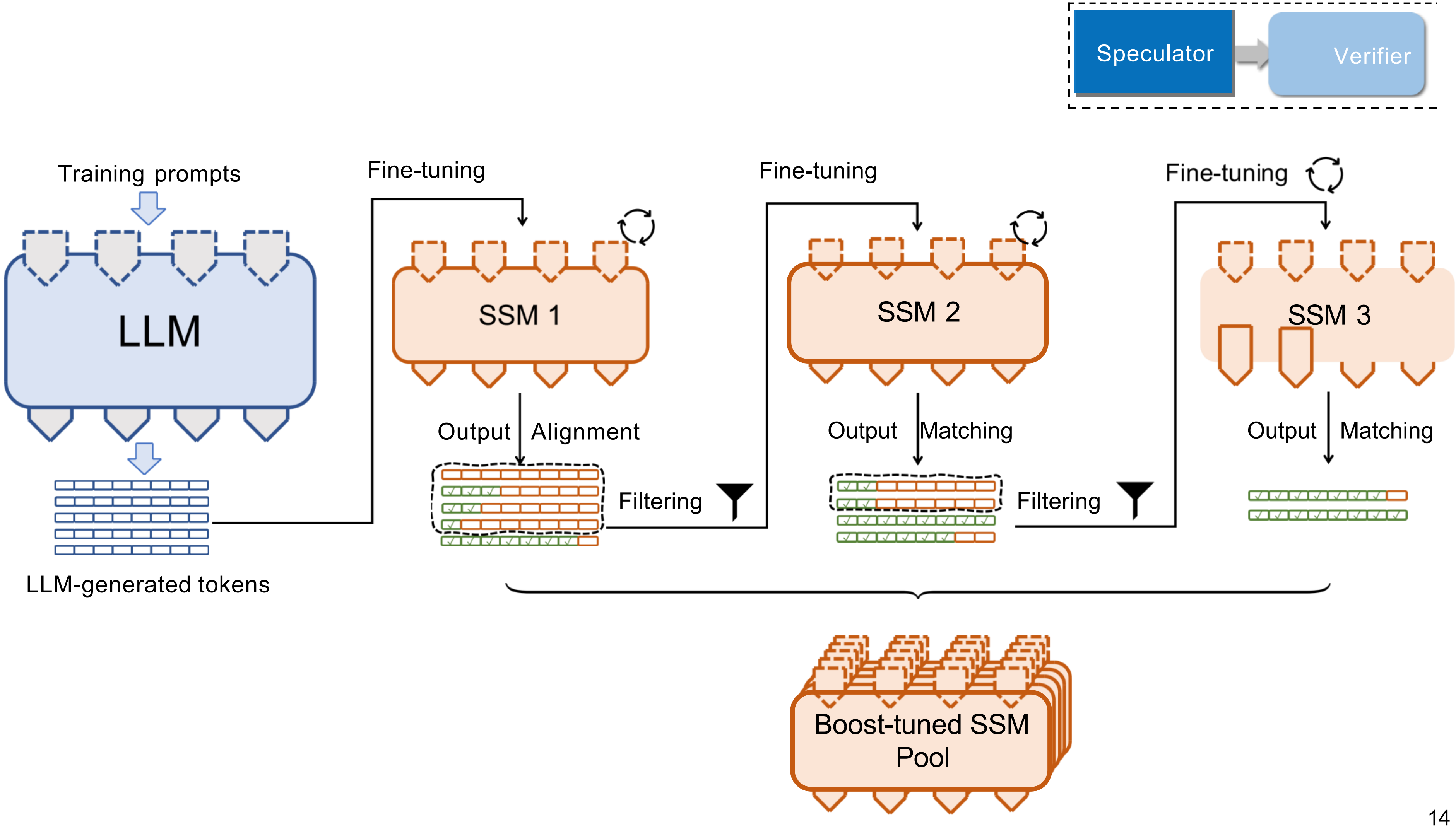


基于学习的Speculator

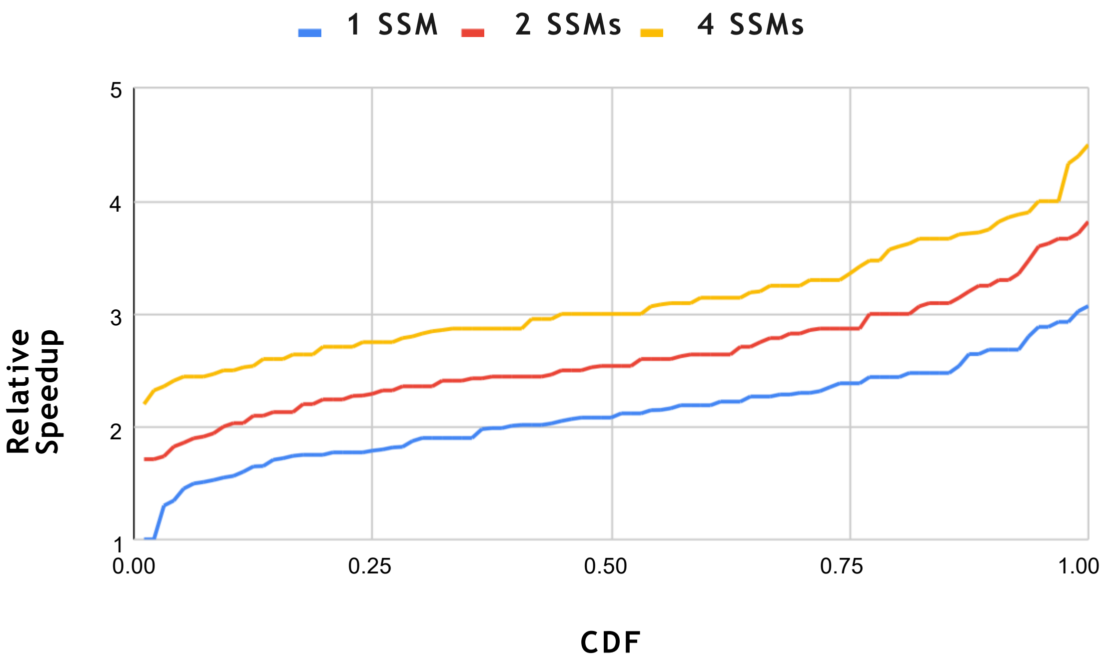




集体增压调谐


Speculator
Verifier
Training prompts
LLM
LLM-generated tokens
Fine-tuning
SSM 1
Output Alignment
Filtering
Fine-tuning
SSM 2
Output Matching
Filtering
Fine-tuning
SSM 3
Output Matching
Boost-tuned SSM Pool

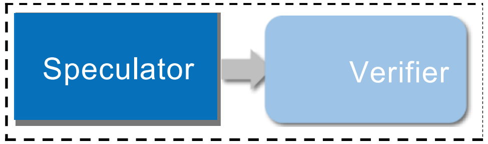
集体增压调校持续提升性能

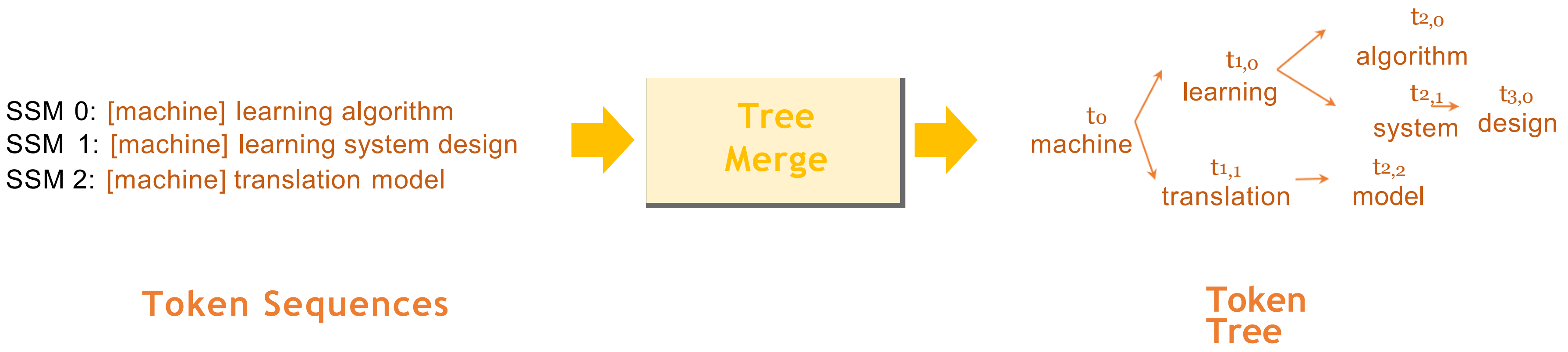


* LLM: LLAMA-7B, SSM: LLAMA-160M, dataset: Alpaca, GPU: NVIDIA A10

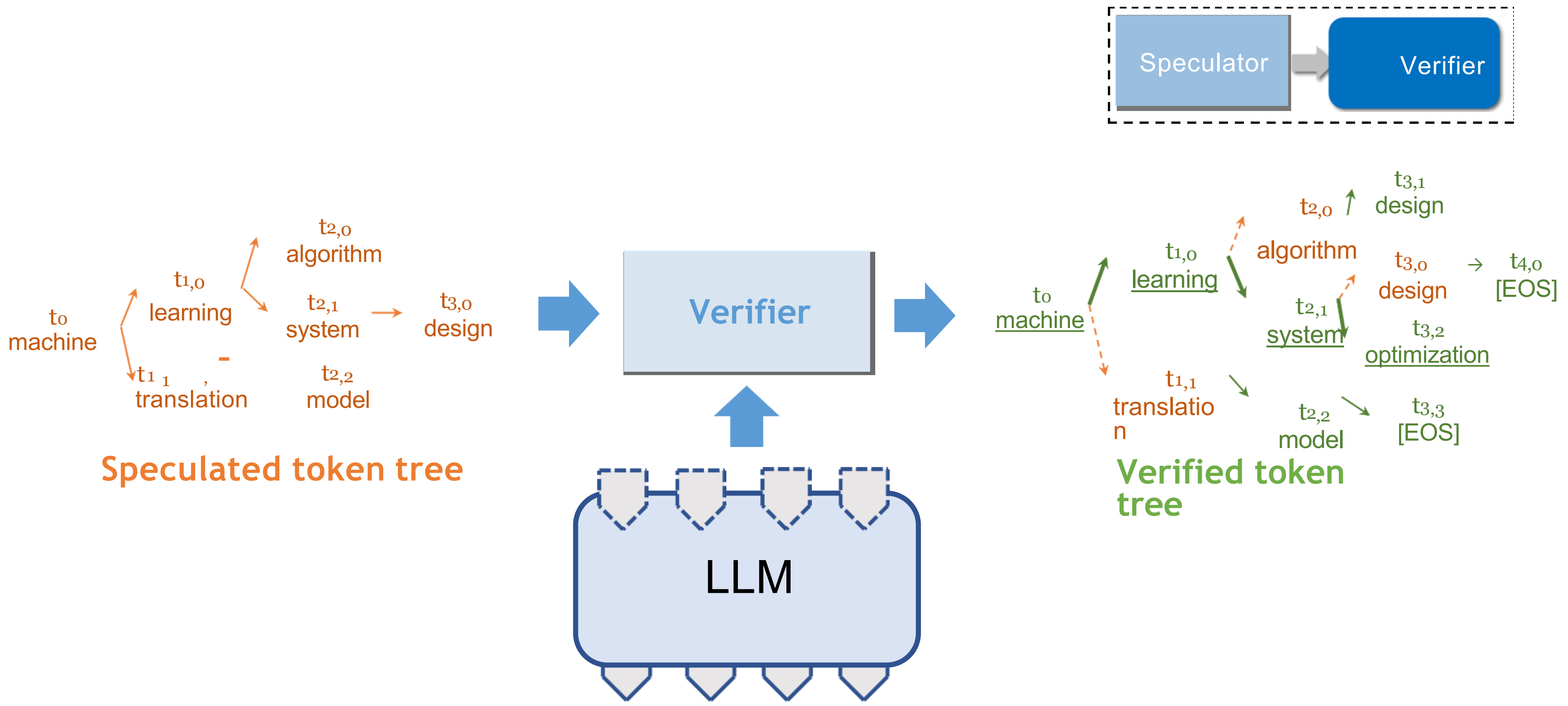
token树合并

- 一种简洁的方式来表示推测token



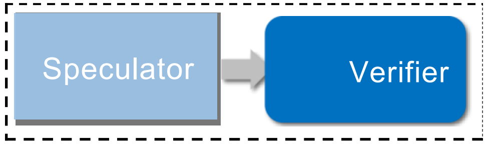


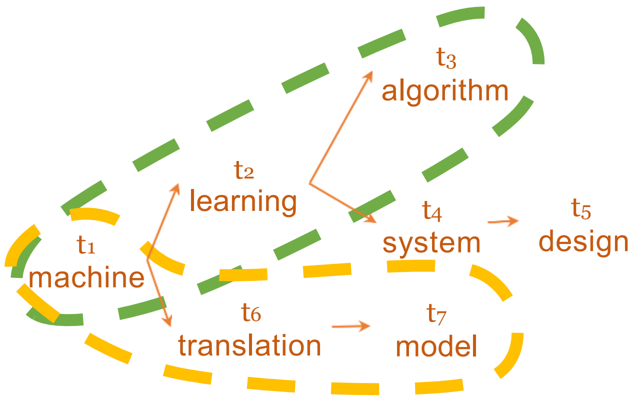
token树验证器

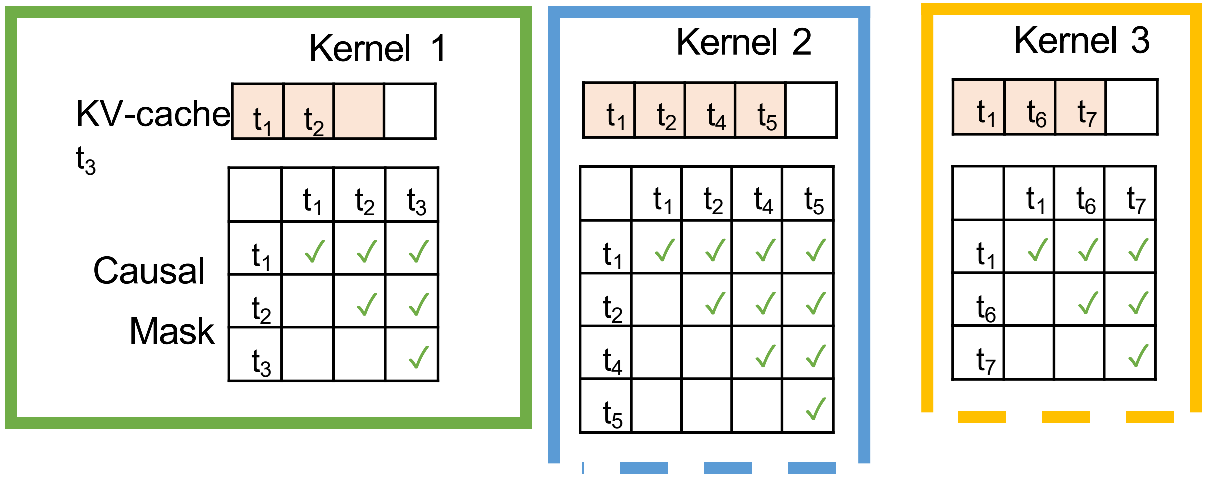


Speculator
Verifier
t0
machine
t1,0
learning
t1,1
translation
t2,0
algorithm
t2,1
system
t2,2
model
t3,0
design
Speculated token tree
Verifier
LLM
t0
machine
t1,0
learning
t1,1
translatio
n
t2,0
algorithm
t2,1
system
t2,2
model
t3,1
design
t3,0
design
t3,2
optimization
t3,3
[EOS]
t4,0
[EOS]
Verified token
tree

基于序列的译码





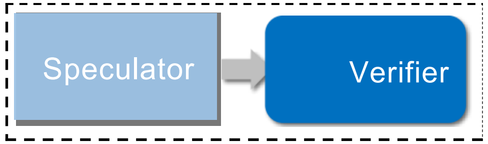


议题:

•冗余解码计算

•更多的请求→更多的GPU内存用于键值缓存

树注意力

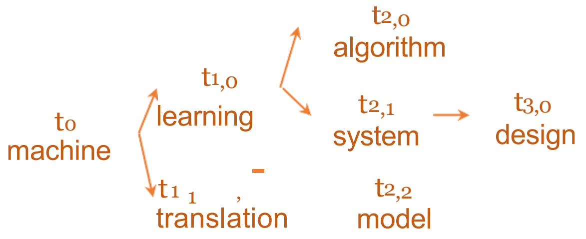


•每个符号的序列注意力输出相同;无冗余

Token Sequences

Attention([machine] learning algorithm
[machine] learning system design
[machine] translation model)

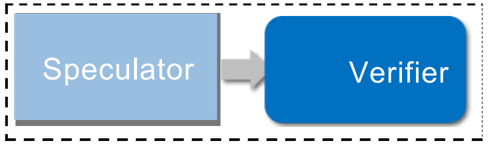
= Tree-Attention(

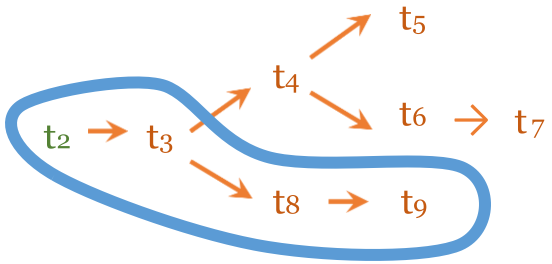


)

Token Tree

基于树的并行解码





KV-cache

t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	…

Topology-aware causal mask

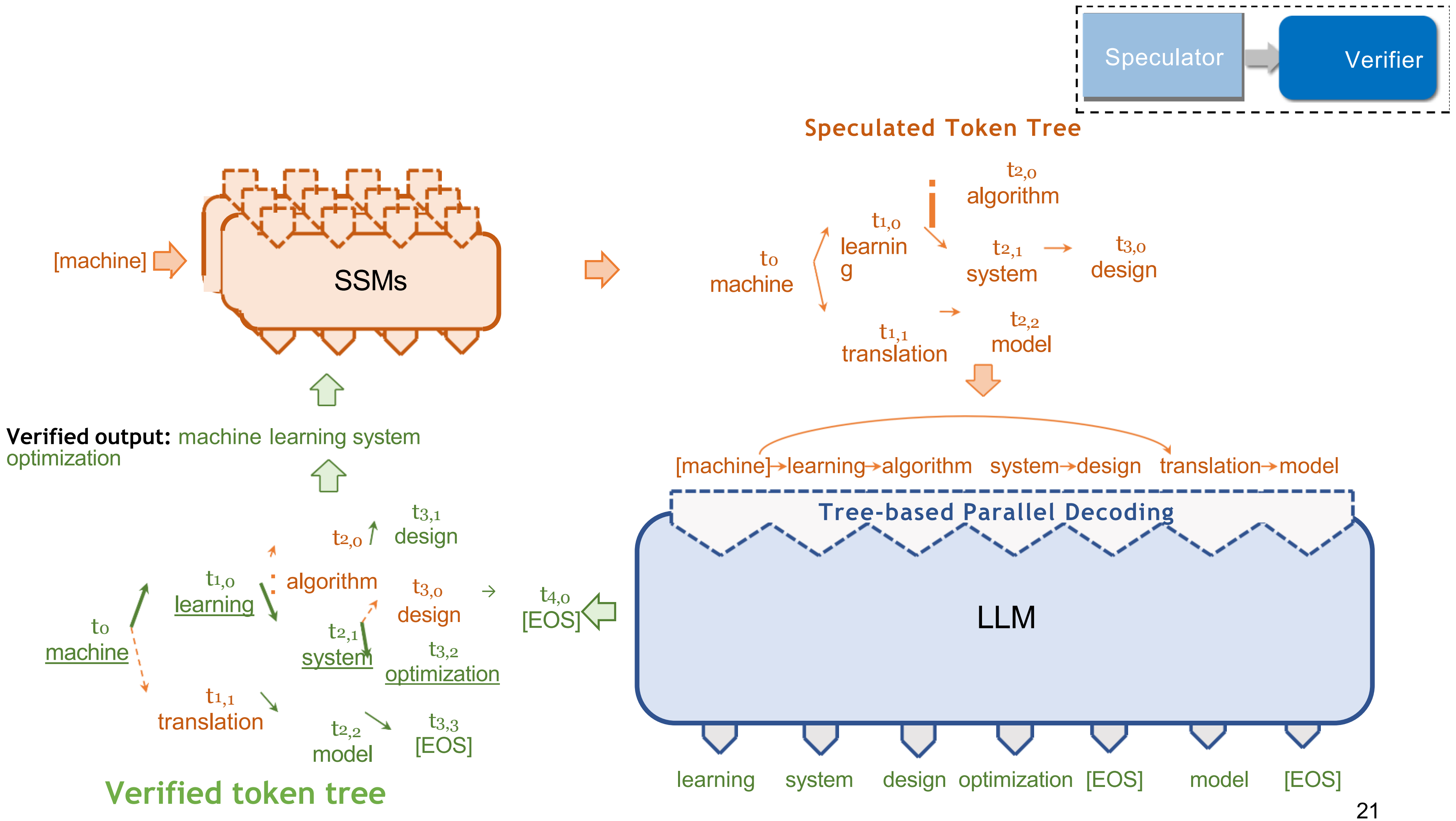
	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9
t_2	✓	✓	✓	✓	✓	✓	✓	✓
t_3		✓	✓	✓	✓	✓	✓	✓
t_4			✓	✓	✓	✓		
t_5				✓				
t_6					✓	✓		
t_7						✓		
t_8							✓	✓
t_9								✓

关键优化:

- 基于DFS的token树线性化方法
- 树拓扑感知因果掩码
- 在单个GPU内核中解码所有token

* SpecInfer: Accelerating LLM Serving with Tree-based Speculative Inference and Verification. ASPLOS’24

验证工作流程



Speculator
Verifier
Speculated Token Tree
[machine]
SSMs
t_0 machine
$t_{1,0}$ learning
$t_{1,1}$ translation
$t_{2,0}$ algorithm
$t_{2,1}$ system
$t_{2,2}$ model
$t_{3,0}$ design
[machine]→learning→algorithm system→design translation→model
Tree-based Parallel Decoding
LLM
learning system design optimization [EOS] model [EOS]
Verified output: machine learning system optimization
t_0 machine
$t_{1,0}$ learning
$t_{1,1}$ translation
algorithm
$t_{2,0}$
$t_{2,1}$ system
$t_{2,2}$ model
$t_{3,1}$ design
$t_{3,0}$ design
$t_{3,2}$ optimization
$t_{3,3}$ [EOS]
$t_{4,0}$ [EOS]
Verified token tree

随机解码

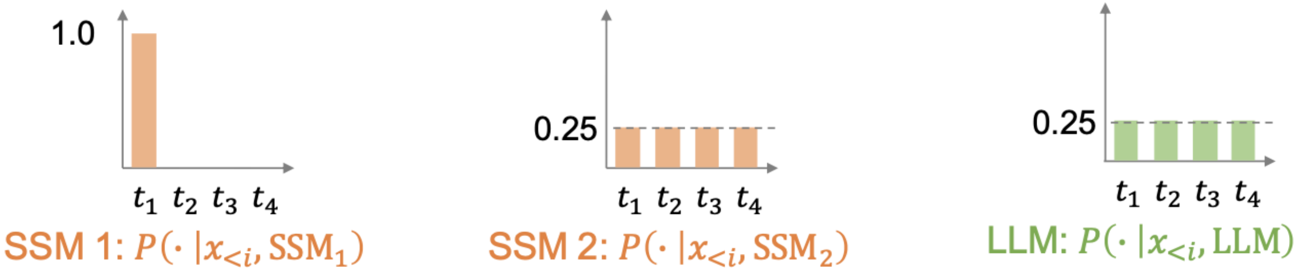
- 挑 战 :验证随机等价性

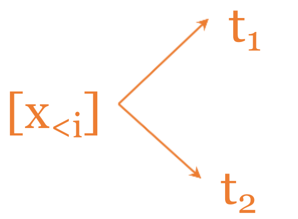
$P_{IncrDecode}(\cdot \mid x_{<i}, LLM) = P_{SpecInfer}(\cdot \mid x_{<i}, LLM, \{SSM_i\})$

- 稻草人方法:朴素采样
  - 用LLM采样 $x_i \sim P_{IncrDecode}(\cdot \mid x_{<i}, LLM)$
  - 确认xi否在token树中

Naïve Sampling can be Suboptimal

- Assume one LLM, two SSMs, and four possible tokens: t_1, t_2, t_3, t_4





Token Tree

朴素抽样的验证概率. = 50%

但我们可以通过直接接受SSM 2做得更好；验证概率. = 100%

关键问题：朴素抽样忽略了P ． $x_{<i}$, SSM以及P ． $x_{<i}$, LLM

推测抽样

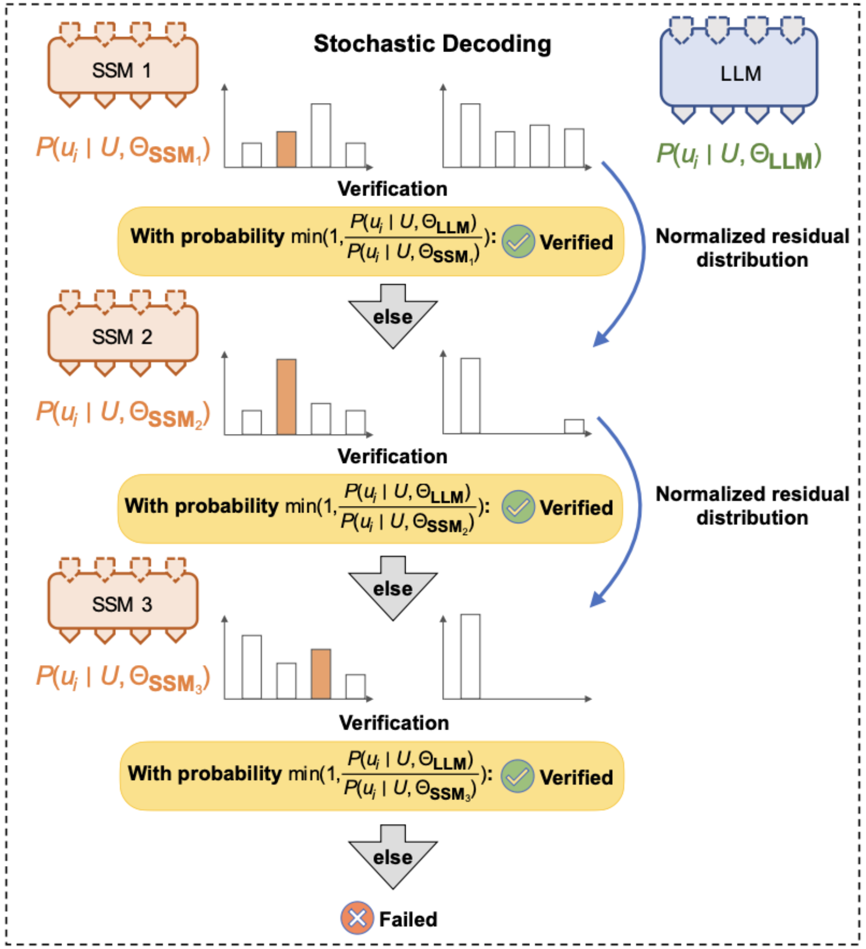
1.使用 SSM 抽样一个token

2. 如果 $P(x|U,\Theta_{SSM}) \le P(x|U,\Theta_{LLM})$,直接接受

3.如果 $P(x|U,\Theta_{SSM}) > P(x|U,\Theta_{LLM})$,接受X，概率. $\frac{P(x|U,\Theta_{LLM})}{P(x|U,\Theta_{SSM})}$

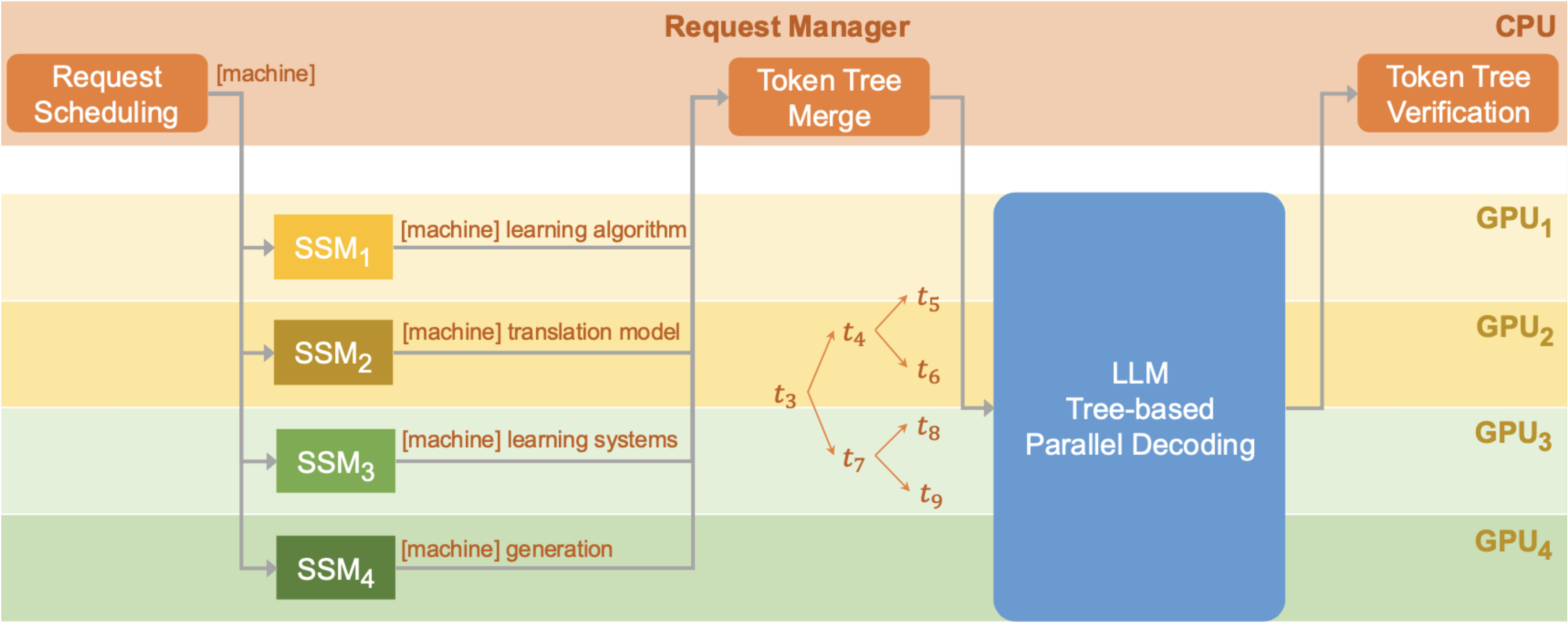
4.如果拒绝X，则归一化残差分布

$$P'(x|U,\Theta_{LLM}) = norm(\max(0, P(x|U,\Theta_{LLM}) - P(x|U,\Theta_{SSM})))$$



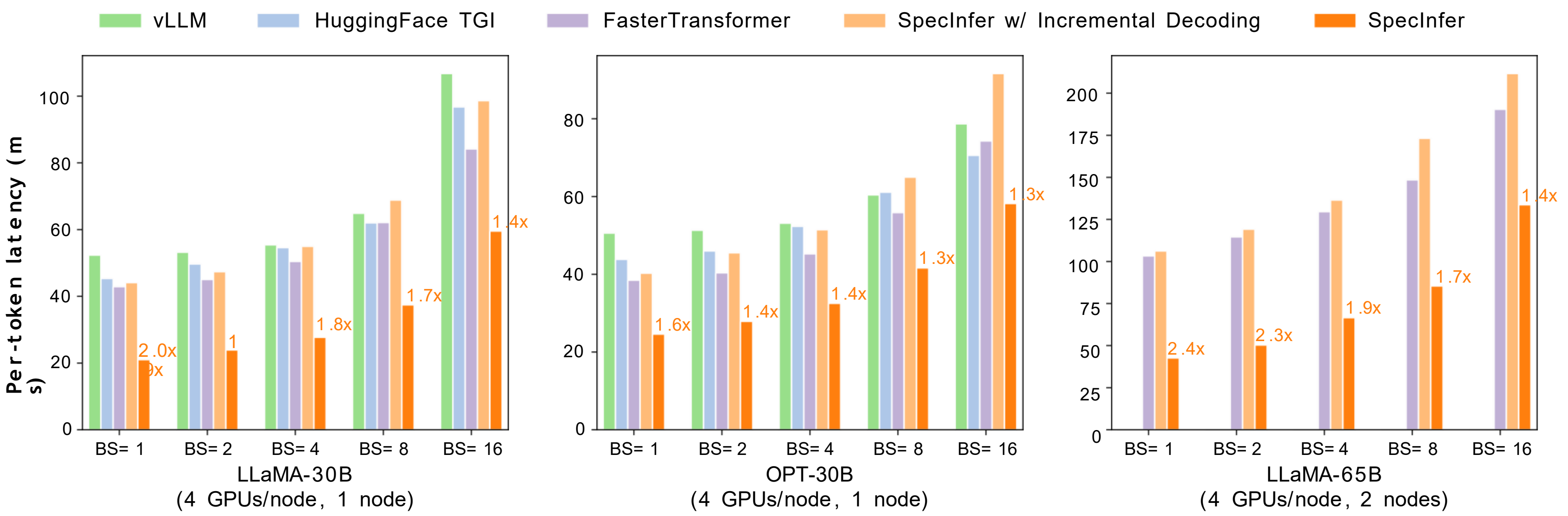
* Details in SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification. ASPLOS,24

分布式LLM服务

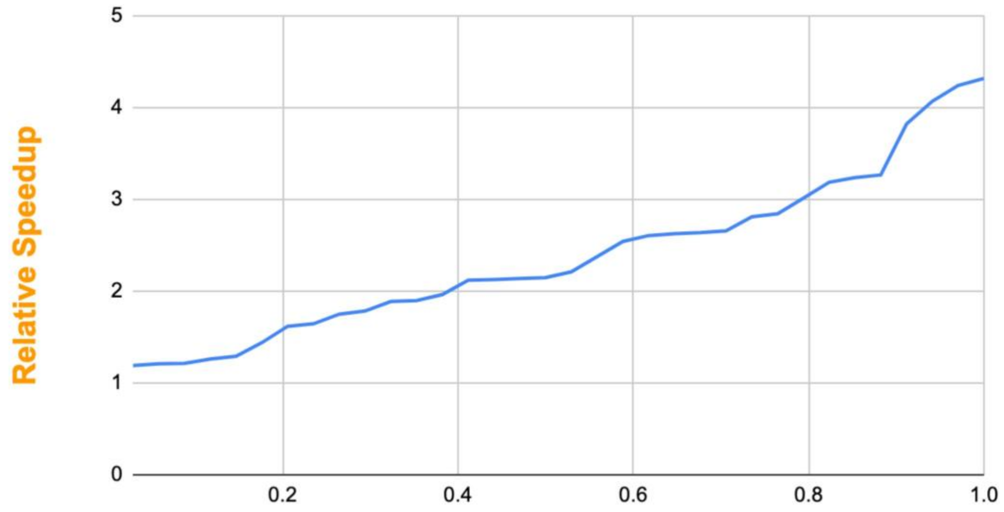


Request Manager
CPU
Request Scheduling
[machine]
Token Tree Merge
Token Tree Verification
GPU$_1$
GPU$_2$
GPU$_3$
GPU$_4$
SSM$_1$
[machine] learning algorithm
SSM$_2$
[machine] translation model
SSM$_3$
[machine] learning systems
SSM$_4$
[machine] generation
t_3
t_4
t_5
t_6
t_7
t_8
t_9
LLM
Tree-based
Parallel Decoding

SpecInfer 将 LLM 推理加速 1.3-2.4 倍

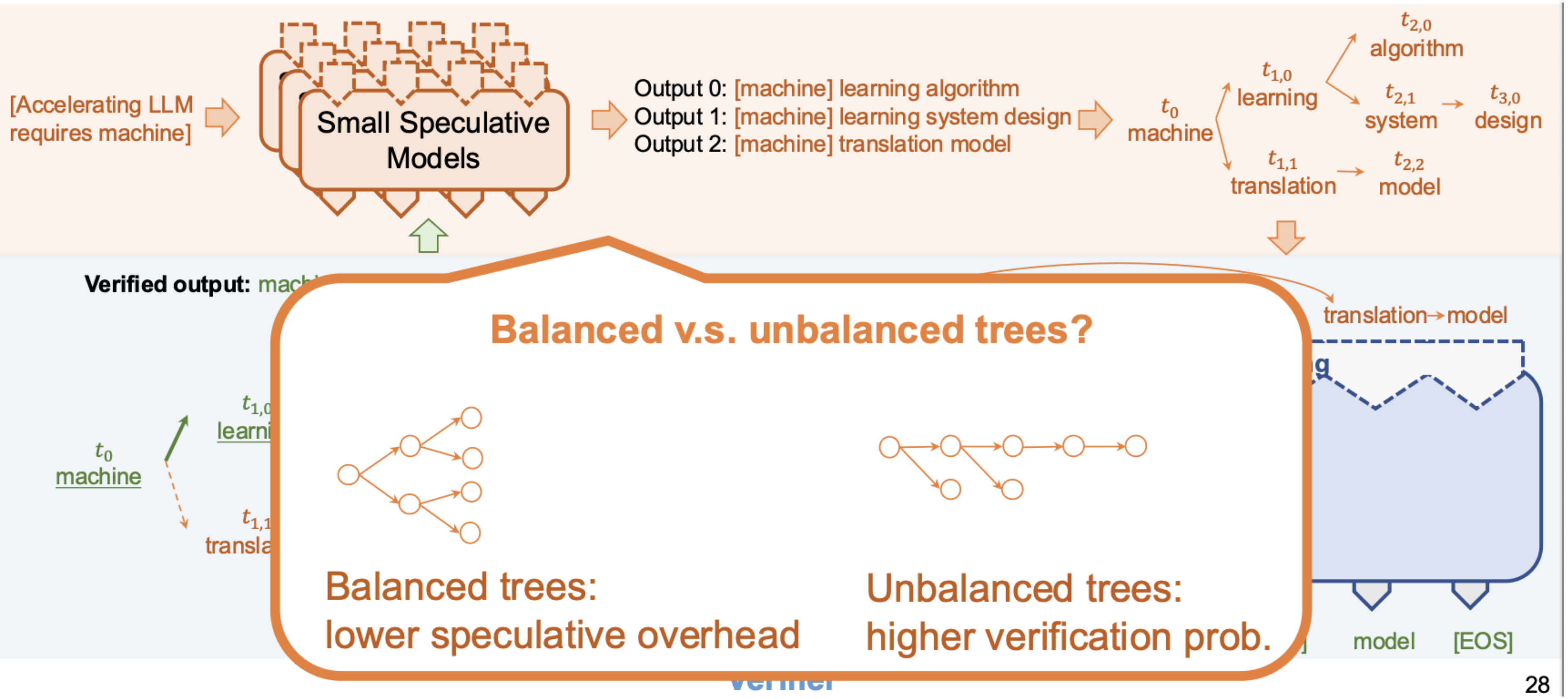


SpecInfer 可以持续加速 LLM 推断



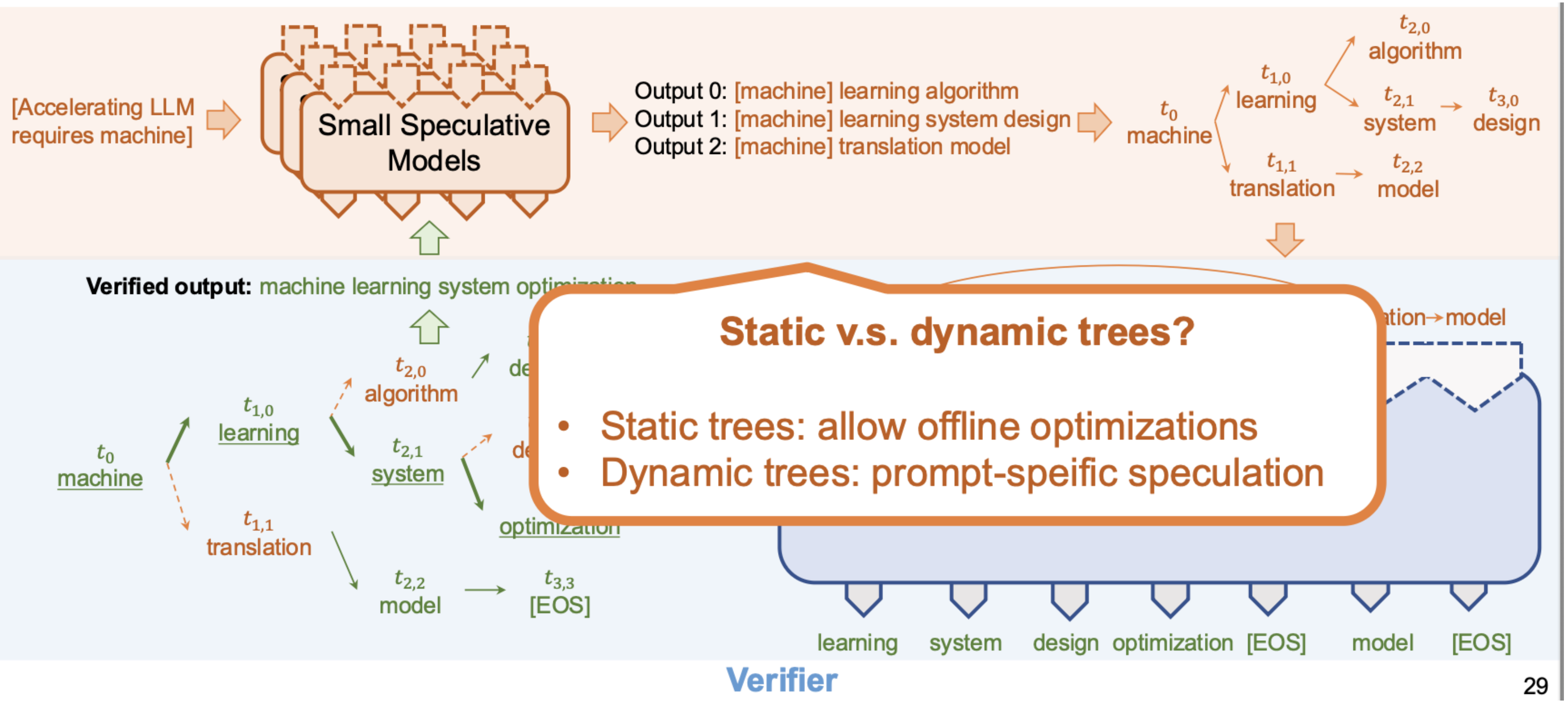
LLM: LLaMA-65B, SSMs: LLaMA-160M

开放研究问题



[Accelerating LLM requires machine]
Small Speculative Models
Output 0: [machine] learning algorithm
Output 1: [machine] learning system design
Output 2: [machine] translation model
t_0 machine
$t_{1,0}$ learning
$t_{1,1}$ translation
$t_{2,0}$ algorithm
$t_{2,1}$ system
$t_{2,2}$ model
$t_{3,0}$ design
Verified output:
translation→model
model
[EOS]
Balanced v.s. unbalanced trees?
Balanced trees:
lower speculative overhead
Unbalanced trees:
higher verification prob.

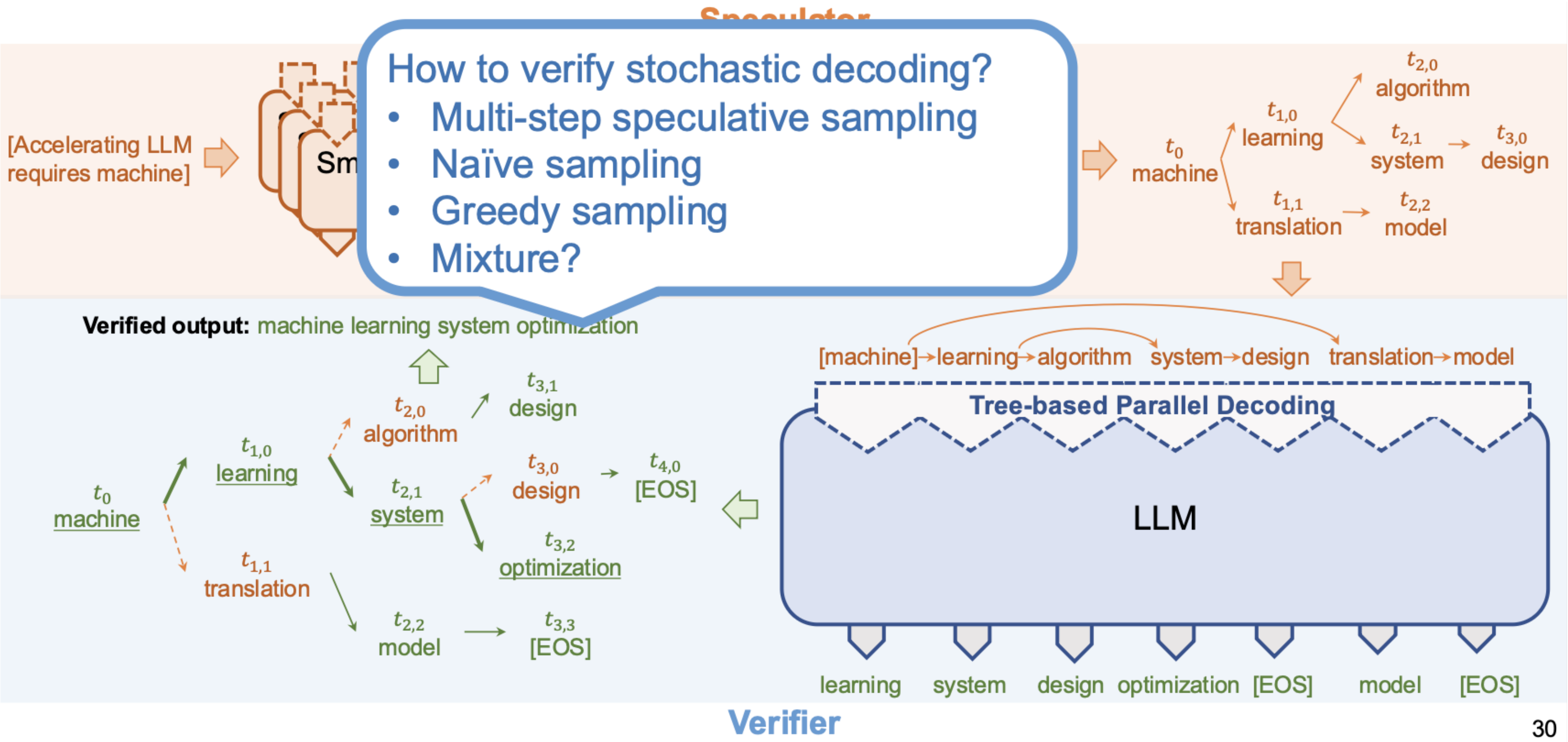
开放研究问题



Static v.s. dynamic trees?

- Static trees: allow offline optimizations
- Dynamic trees: prompt-speific speculation

开放研究问题

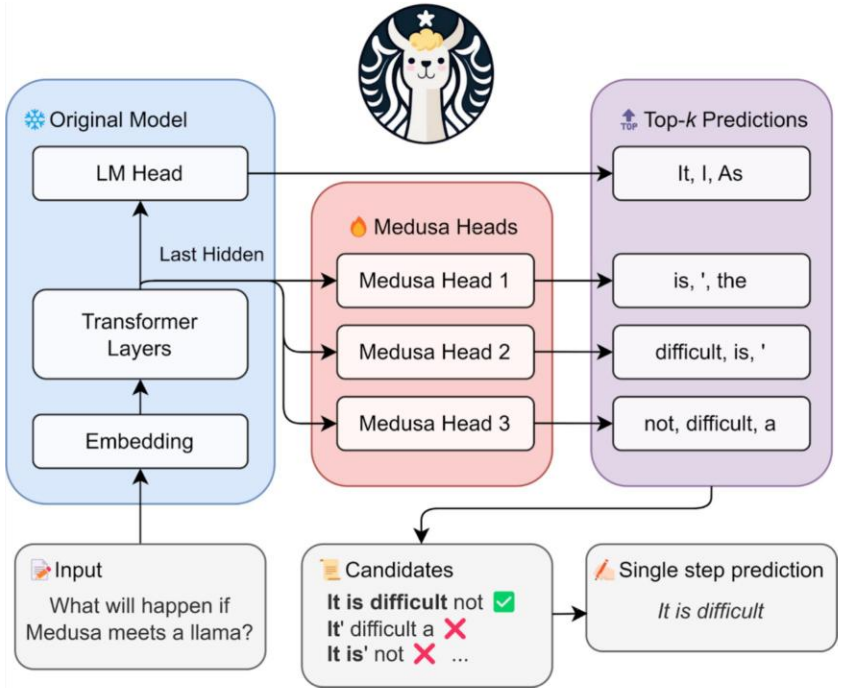


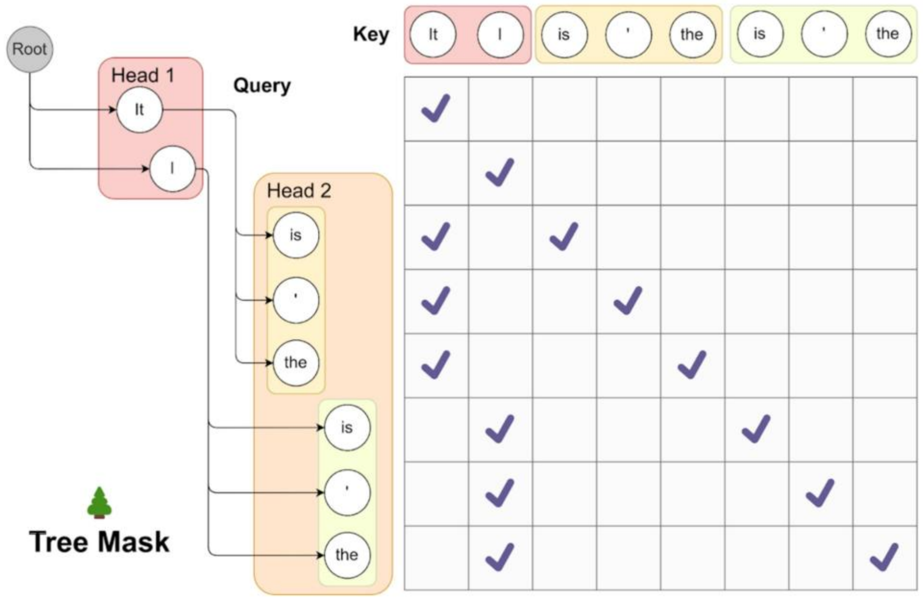
How to verify stochastic decoding?

- Multi-step speculative sampling
- Naïve sampling
- Greedy sampling
- Mixture?

美杜莎：多解码头下的推测性解码

•引入可训练的注意力头来预测未来token



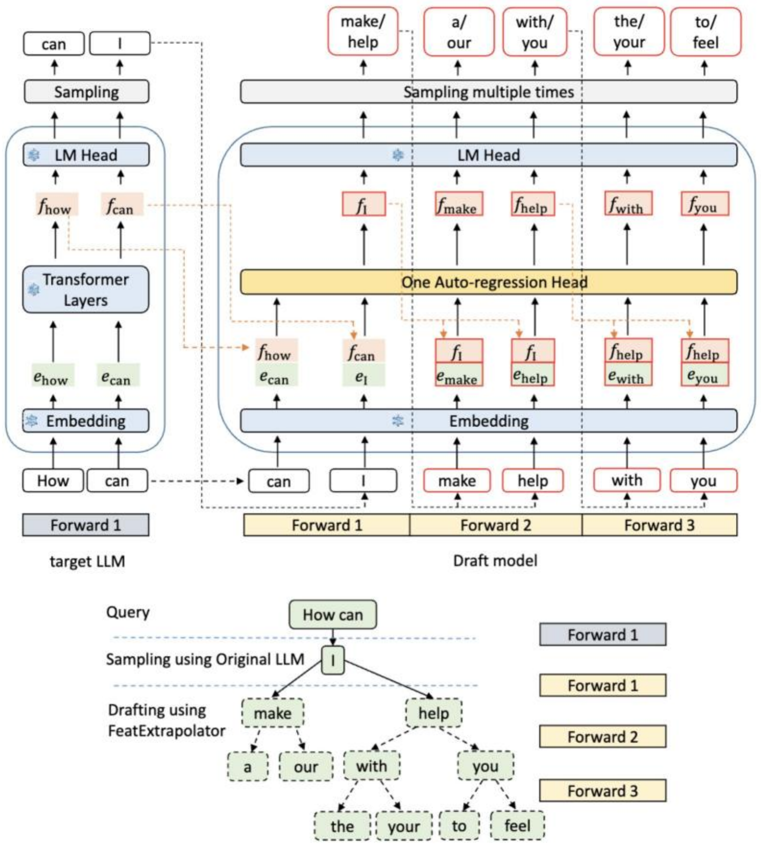


https://www.together.ai/blog/medusa

Eagle:动态投机token树

•草图模型重用嵌入和LLM的LM头层，引入可训练的注意力层

•扩展阶段：草图模型选择K每次迭代中需解码的节点



EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty
EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees

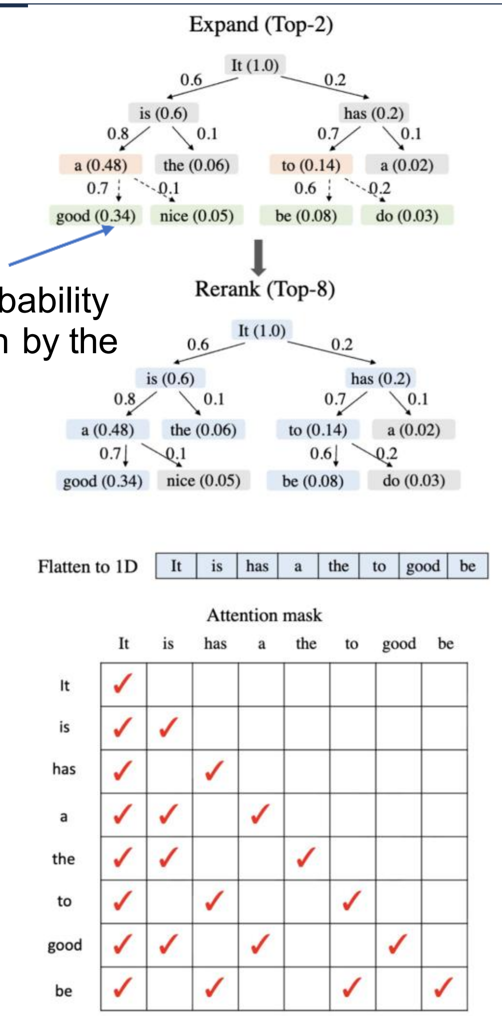
Eagle:动态投机token树

•草图模型重用嵌入和LLM的LM头层，引入可训练的注意力层

•扩展阶段：草图模型选择K每次迭代中需解码的节点

•重新排名阶段：根据累积的可能性选择N个节点进行验证

Accumulated probability to select that path by the draft model



EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty
EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees

目 录

Contents

提示查找解码

- **Prompt:** What is the capital of South Korea?

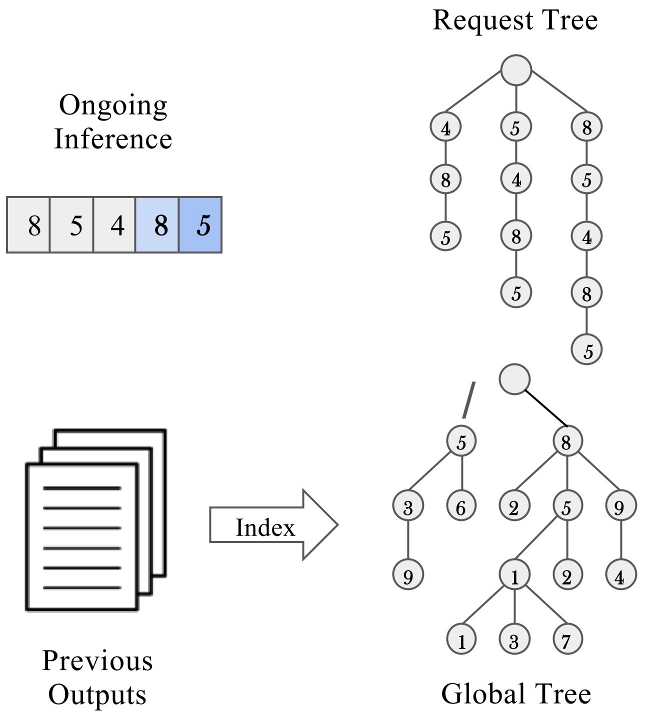
2-gram	3 speculate tokens
What is	the capital of
is the	capital of South
the capital	of South Korea
capital of	South Korea ?

During generation:

the capital of South Korea

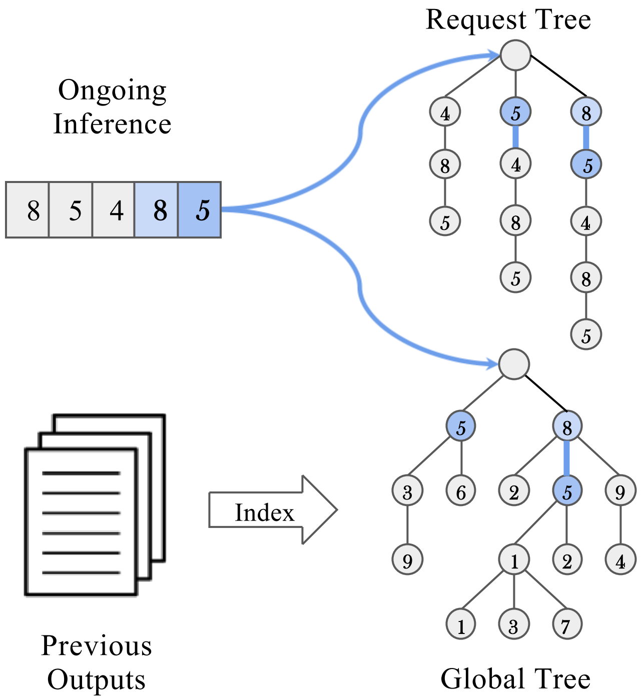
✅ ✅ ✅

SuffixDecoding:二层后缀树的猜测



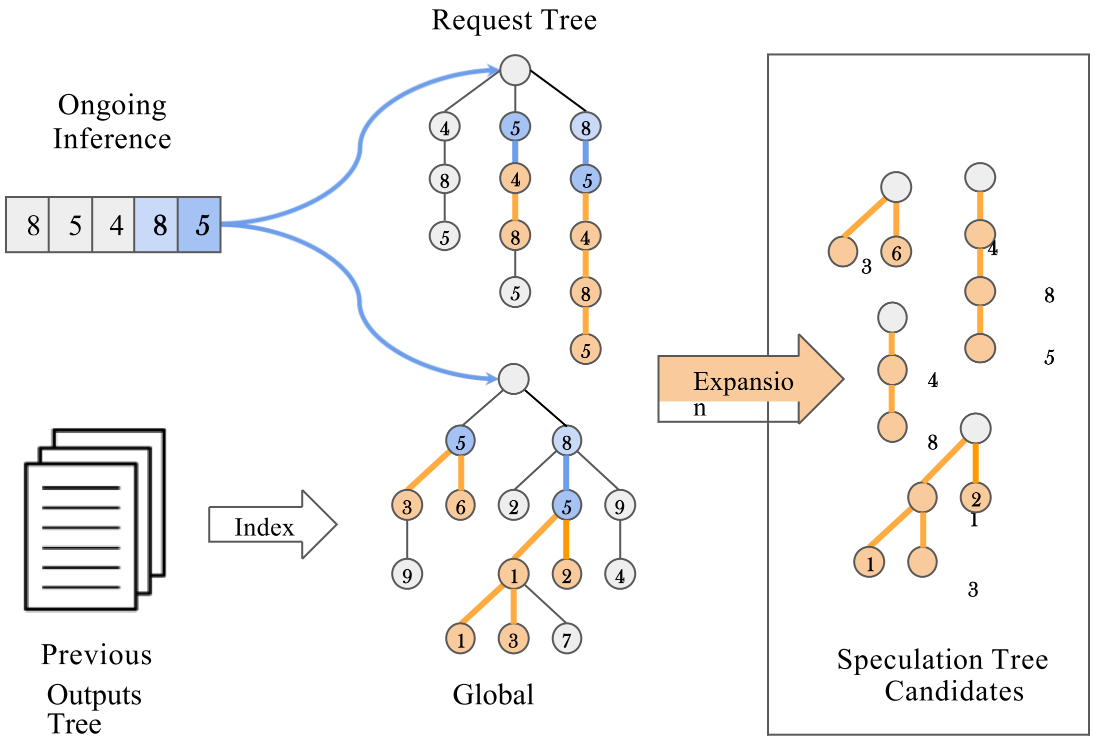
SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications

SuffixDecoding:二层后缀树的猜测



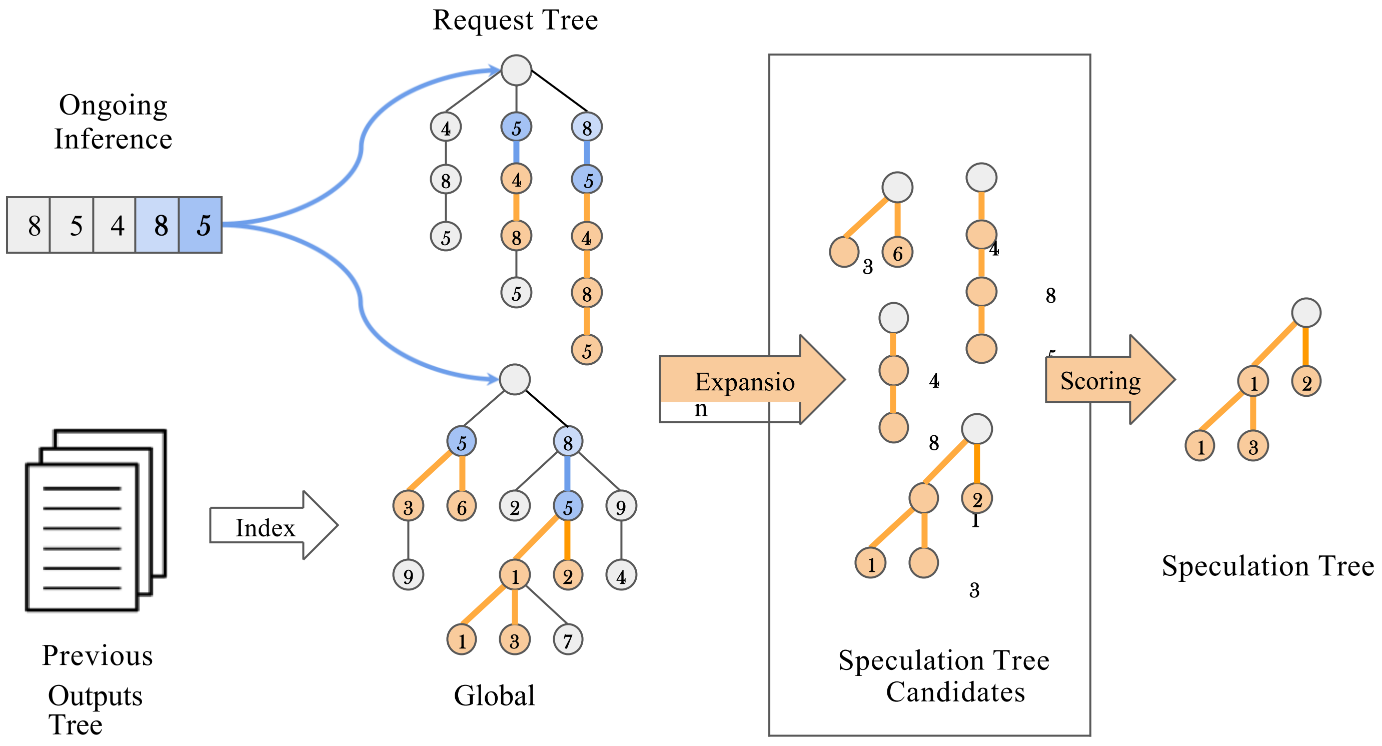
SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications

SuffixDecoding:二层后缀树的猜测



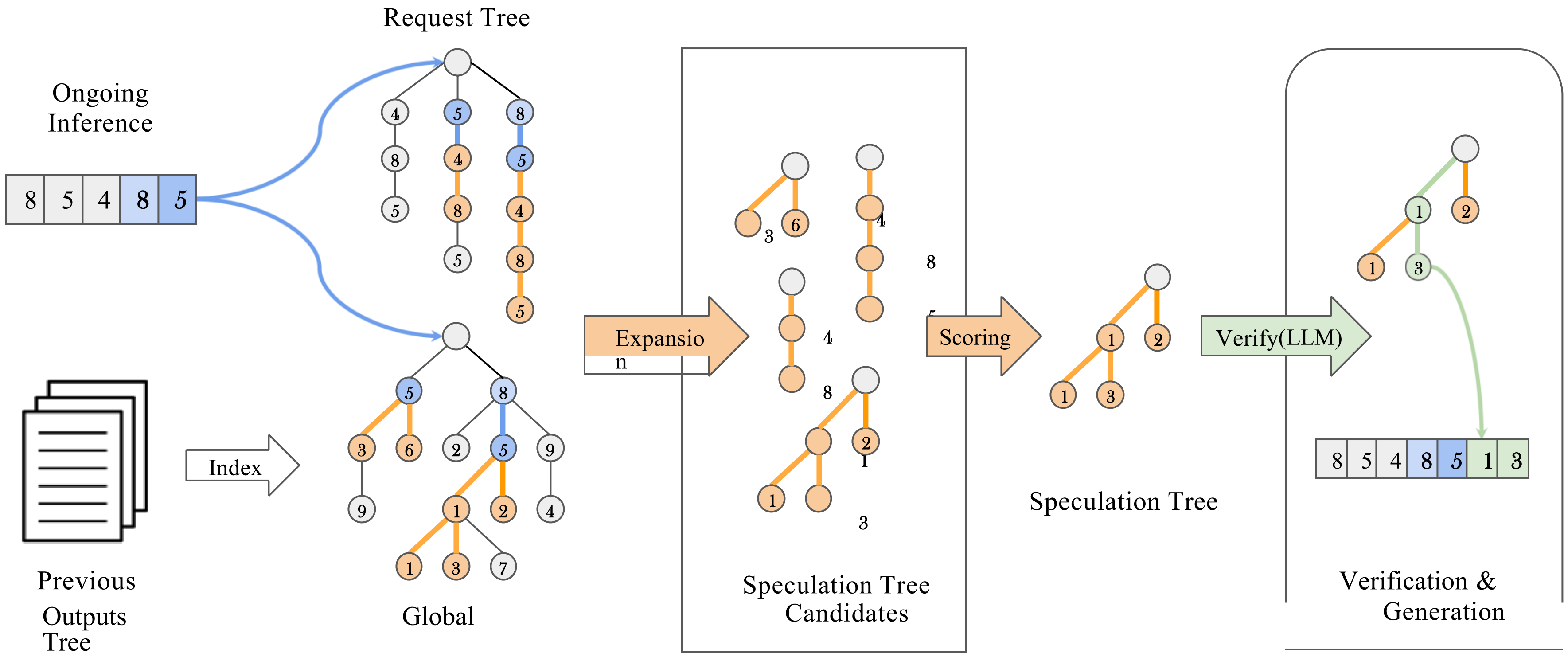
SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications

SuffixDecoding:二层后缀树的猜测



SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications

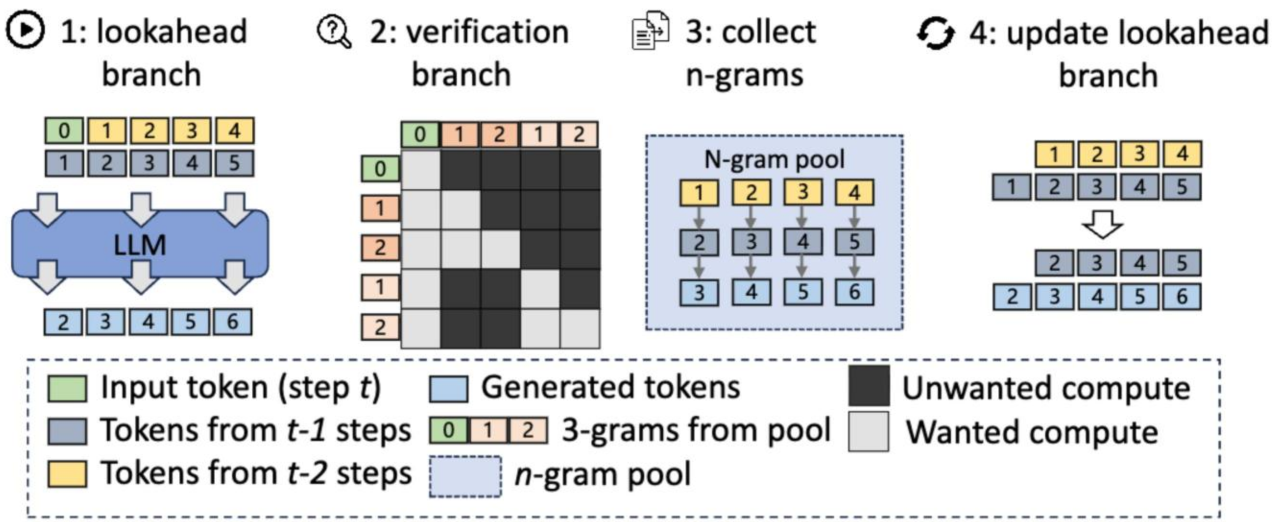
SuffixDecoding:二层后缀树的猜测



SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications

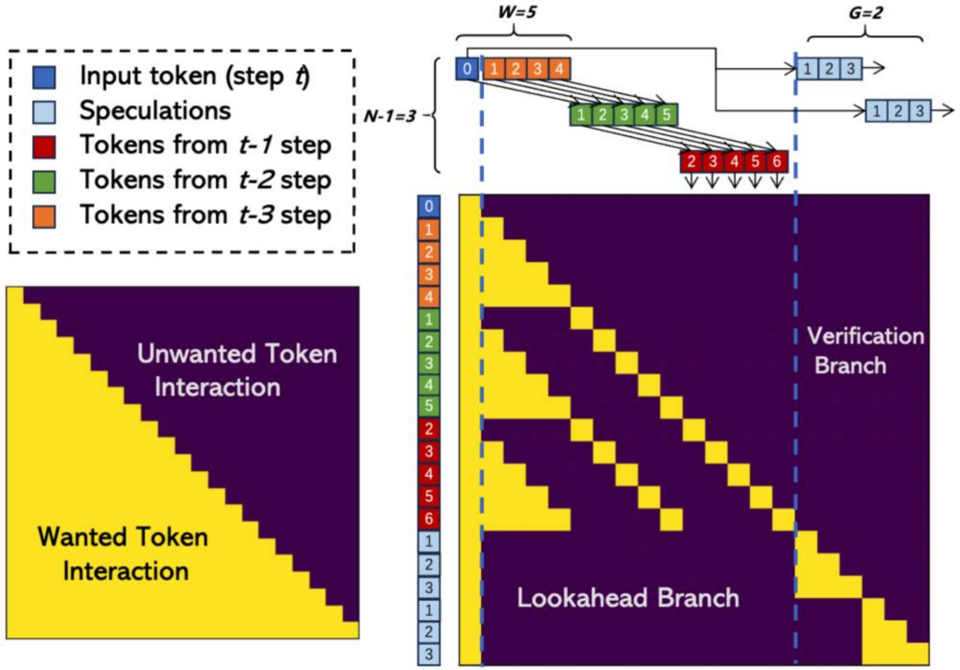
前瞻解码

1.前瞻分支：在每个位置生成一个token
2.验证分支：验证多个3-gram（从3-gram池中搜索）
3.收集并缓存从前瞻分支新生成的3-gram
4.更新lookahead分支



前瞻解码

•批量验证和前瞻分支一次性完成



(a) Causal Mask

(b) Lookahead Decoding Mask

Break the Sequential Dependency of LLM Inference Using Lookahead Decoding